

AI-Powered Search Product Page

Easy Shopping Mall - Feature Proposal

Prepared: July 22, 2026

Status: Ready for Review

Executive Summary

Implement an AI-powered product search feature that allows customers to search products using natural language (e.g., "wireless earbuds under \$50 with good reviews") instead of basic keyword search. The system uses Google Gemini (primary) and xAI Grok (fallback) APIs - both available for free.

Problem Statement

Current Limitations

- Basic keyword search requires exact matches
- Customers must know product names or categories
- No understanding of intent (e.g., "gift for mom" won't find relevant products)
- Limited filtering via manual dropdowns

Customer Pain Points

- "I want running shoes but don't know the exact brand name"
- "Show me products under 2000 taka for women"
- "Find me something red in size M"

Proposed Solution

AI-Powered Natural Language Search

The system works in 4 simple steps:

1. Customer types natural language query (e.g., "women's red running shoes under 2000 taka")
2. AI (Gemini/Grok) parses the intent into structured MongoDB filters
3. MongoDB query executes on existing Product collection
4. Returns matching products to customer

Technical Architecture

System Components

Next.js Frontend	Node.js/Express	MongoDB
Search Page	AI Service API	Product Collection
User Input	Query Parser	Indexed Fields
Results Display	Error Handler	Cached Results

Data Flow

Step	Component	Action
1	Frontend	Customer enters natural language query
2	Backend	Receives query, sends to AI API
3	AI API	Parses query into structured MongoDB filters
4	Backend	Executes MongoDB query with parsed filters
5	Frontend	Displays matched products

AI Provider Comparison

Feature	Gemini (Primary)	Grok (Fallback)
Free Tier	15 RPM, 1M tokens/day	Limited requests
Model	gemini-1.5-flash	grok-beta
Cost	\$0 (generous free tier)	\$0 (limited free tier)
Reliability	High (Google)	Medium
Setup	ai.google.dev	console.x.ai

Supported Search Fields

The AI can parse natural language into queries for these existing product fields:

Field	Example Query	MongoDB Filter
productName	"iPhone case"	{ productName: /iPhone case/i }
price	"under 1000 taka"	{ price: { \$lte: 1000 } }
color	"red shoes"	{ color: "red" }
gender	"women's clothing"	{ gender: "women" }
brand	"Nike sneakers"	{ brand: "Nike" }
tags	"wireless bluetooth"	{ tags: { \$in: [...] } }
productSize	"size large"	{ productSize: "L" }
ratings	"4 star and above"	{ ratings: { \$gte: 4 } }
discount	"on sale"	{ discount: { \$gt: 0 } }

Files to Create/Modify

New Files

File	Purpose
src/services/aiSearchService.ts	AI API integration (Gemini + Grok)
src/config/ai.ts	AI configuration and API keys

Modified Files

File	Changes
product.controllers.ts	Add aiSearchController function
product.routes.ts	Add POST /product/ai-search route
.env	Add GEMINI_API_KEY and GROK_API_KEY

Implementation Phases

Phase 1: Backend AI Service (2-3 hours)

- Create aiSearchService.ts with Gemini integration
- Add Grok as fallback provider
- Create AI prompt template with product schema context
- Add error handling and rate limit management

Phase 2: API Endpoint (1-2 hours)

- Add aiSearchController to product controllers
- Add route to product routes
- Add input validation
- Test with sample queries

Phase 3: Frontend Integration (2-3 hours)

- Create search input component with debouncing
- Add loading states and error handling
- Display AI-parsed query (optional)
- Mobile responsive design

Phase 4: Testing & Optimization (1-2 hours)

- Test various natural language queries
- Handle edge cases (empty results, API failures)
- Add caching for frequent queries
- Monitor API usage

Total Estimated Time: 6-10 hours

Cost Analysis

Item	Cost
Gemini API (free tier)	\$0
Grok API (free tier)	\$0
Additional hosting	\$0
Total	\$0

Free tier is sufficient for most small-to-medium e-commerce sites.

Example Queries & Results

Customer Types	AI Parsed Query	Results
"cheap phones under 5000"	{ price: { \$lte: 5000 } }	Budget phones
"women's summer dress"	{ gender: "women" }	Summer dresses
"Nike shoes size 10"	{ brand: "Nike" }	Nike shoes
"gift for husband"	{ tags: ["gift", "men"] }	Gift items
"kitchen discount"	{ discount: { \$gt: 0 } }	Sale items

Risk Assessment

Risk	Likelihood	Impact	Mitigation
API rate limiting	Medium	Low	Fallback to Grok then basic search
AI misinterprets	Low	Medium	Show parsed query for transparency
API downtime	Low	Medium	Cache results, fallback to basic
Cost overrun	Very Low	High	Free tier is generous

Benefits

For Customers

- Natural language search
- Better product discovery
- Faster search
- More intuitive filtering

For Business

- Increased conversion rates
- Better user experience
- Competitive advantage
- Low cost (\$0)

For Development

- No database schema changes
- Reuses existing data
- Simple integration
- Easy to maintain

Success Metrics

Metric	Current	Target
Search success rate	~60%	85%+
Avg. search time	2-3s	<1s
Customer satisfaction	-	4.5+/5
Search abandonment	40%	20%

Next Steps

1. Approve proposal - Team lead review
2. Get API keys - Gemini (required) + Grok (optional)
3. Implement backend - AI service + API endpoint
4. Implement frontend - Search UI component
5. Test & deploy - QA + production release

Appendix

Environment Variables Required

```
# AI Search Configuration
GEMINI_API_KEY=your_gemini_api_key_here
GROK_API_KEY=your_grok_api_key_here # Optional
```

Product Schema Reference

```
interface IProduct {  
  productName: string;  
  description: string;  
  category: ObjectId[];  
  subCategory: ObjectId[];  
  brand: string;  
  price: number;  
  dropshippingPrice: number;  
  productStock: number;  
  discount: number;  
  ratings: number;  
  tags: string[];  
  color: string[];  
  productSize: string[];  
  gender: string; // "men" | "women" | "unisex"  
}
```